

****Disclaimer****

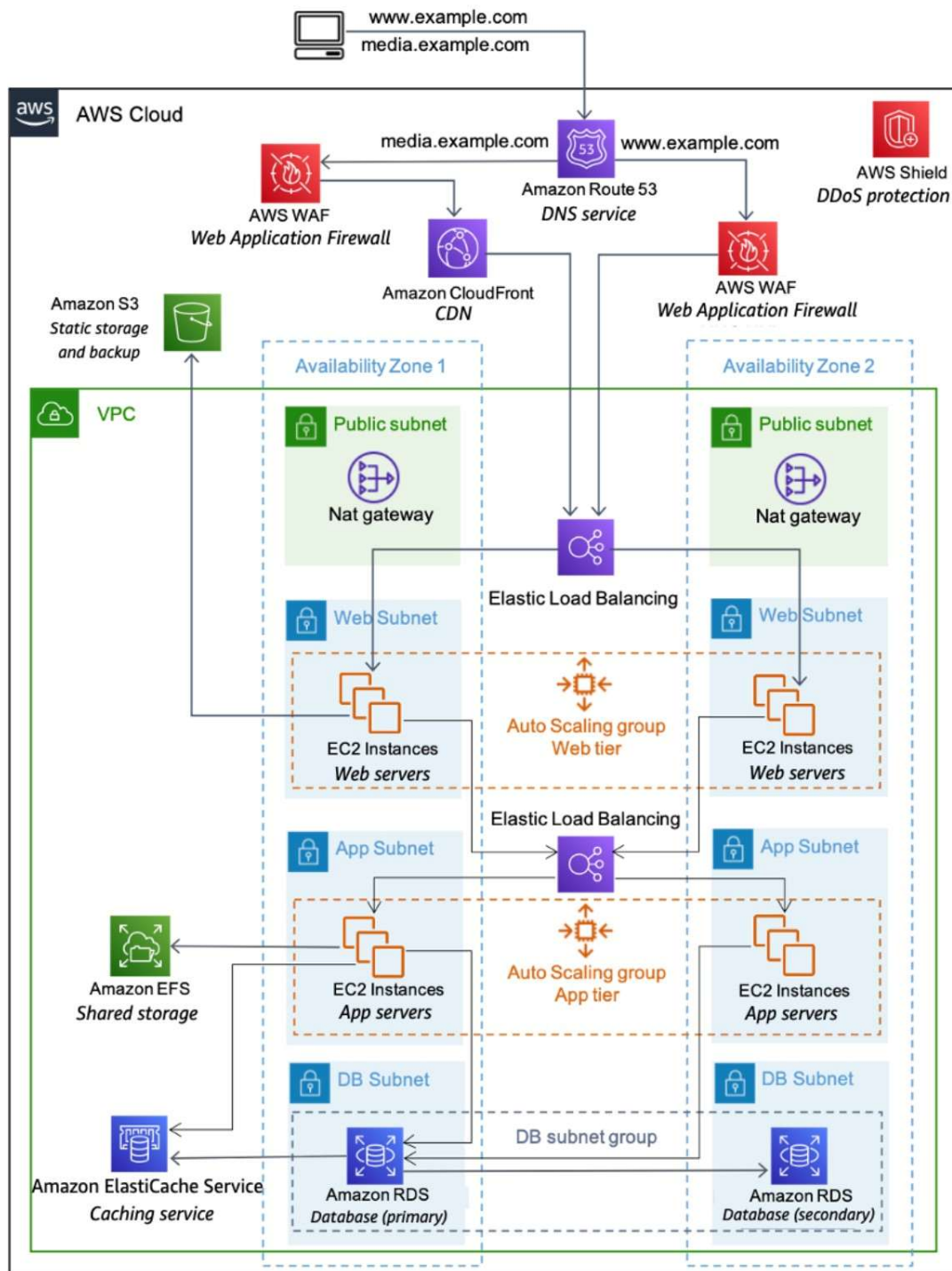
This document has been prepared as part of my personal study journey for the **AWS Certified Solutions Architect – Associate (SAA-C03) ** certification. It represents my understanding of the concepts covered during my studies and is intended for educational and demonstration purposes.

The explanations, notes, diagrams, and overall structure presented in this document have been created and refined by me. Where necessary, I have reorganized, redesigned, and enhanced the presentation of information to improve clarity, readability, and understanding.

However, the AWS web application architecture image included in this document is **not my original work. It has been sourced from publicly available educational and reference materials, and all rights and credit for that specific architecture image belong to its original creator(s). Any AWS concepts discussed are based on official AWS documentation and other educational resources, interpreted and explained in my own words to reflect my learning and understanding.**

Danke!

AWS Three-Tier Highly Available Web Application Architecture



Complete Study Explanation

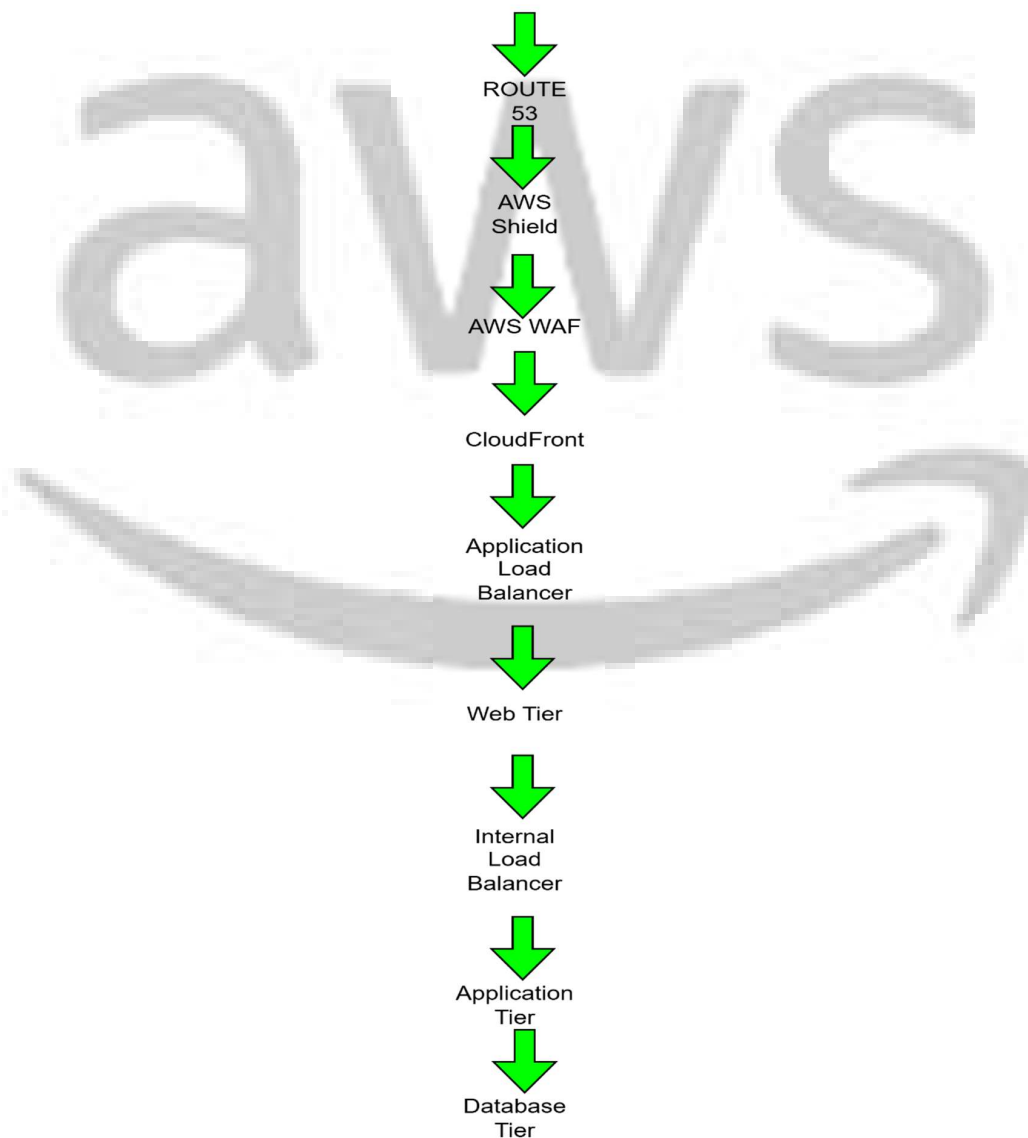
Introduction

This architecture represents a **secure, scalable, highly available, and fault-tolerant** web application deployed in AWS. It follows the **Three-Tier Architecture**, where the application is separated into:

- Presentation Layer (Web Tier)
- Business Logic Layer (Application Tier)
- Data Layer (Database Tier)

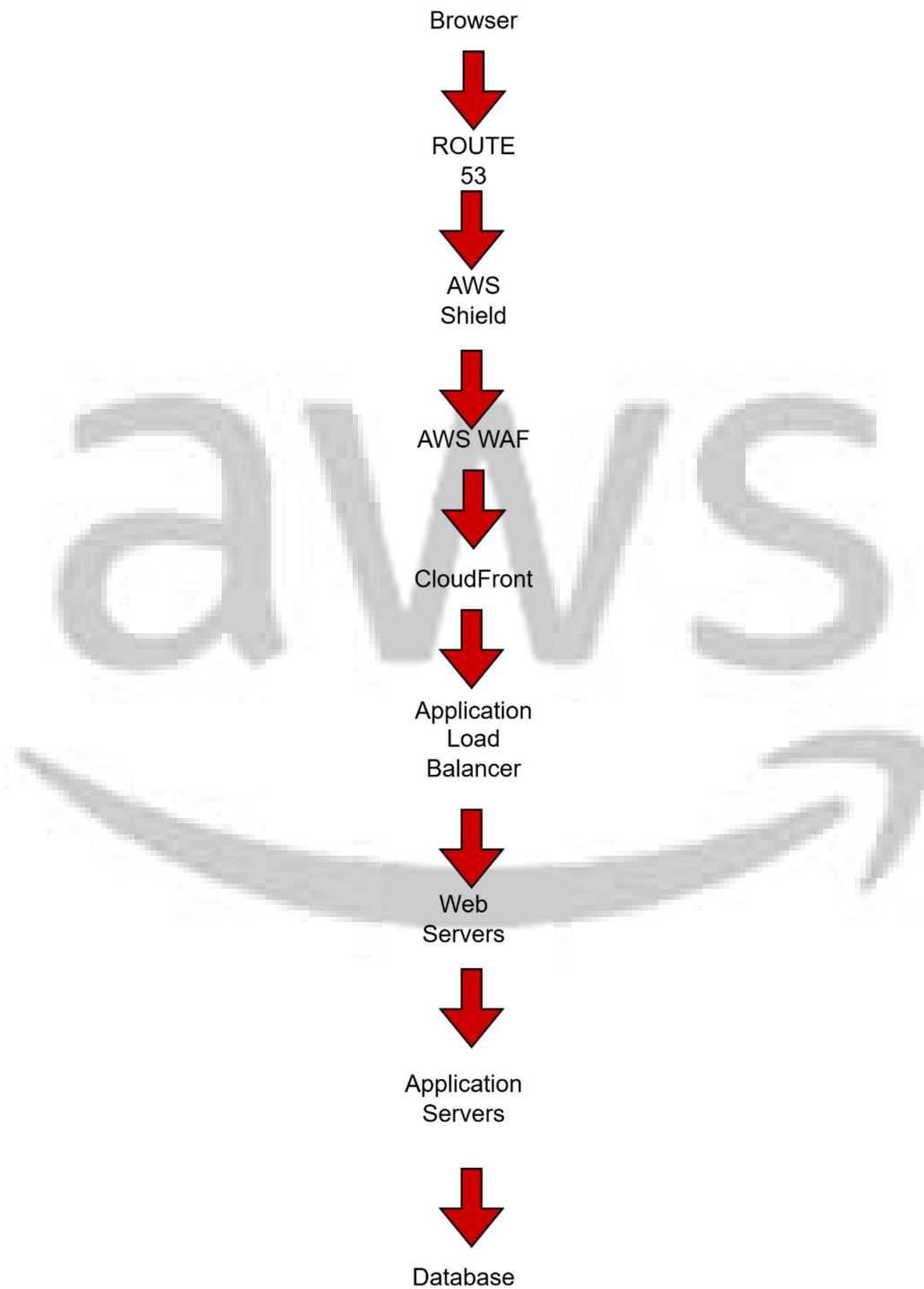
By separating these layers, each component can scale independently, remain secure, and continue operating even if another component fails.

High-Level Architecture



Request Flow

Whenever a user visits the website, the request follows this sequence.



Each AWS service performs a specific task before forwarding the request to the next service.

Stage 1 – User Accesses the Website

Suppose a customer enters <https://www.example.com>. The computer first needs to determine the IP address associated with the domain. This process is called **DNS Resolution**. AWS uses **Amazon Route 53** for DNS management.

Amazon Route 53

Purpose

Amazon Route 53 is AWS's managed DNS service. Its responsibility is to translate www.example.com into an IP address. Without DNS, users would have to remember numeric IP addresses rather than easy-to-read domain names.

Main Features

- Domain registration
- DNS resolution
- Health checks
- Failover routing
- Traffic routing policies

Why Route 53?

If one server becomes unavailable, Route 53 can automatically redirect users to another healthy endpoint.

AWS Shield

Purpose

AWS Shield protects applications against Distributed Denial of Service (DDoS) attacks. A DDoS attack attempts to overwhelm servers by sending millions of fake requests. Instead of allowing these requests to reach the application, AWS Shield filters them first.

Benefits

- Protects Internet-facing applications
- Automatically enabled for many AWS services
- Helps maintain application availability

AWS Web Application Firewall (WAF)

Purpose

While AWS Shield protects against network attacks, AWS WAF protects against attacks targeting the application itself.

WAF Blocks

- SQL Injection
- Cross-Site Scripting (XSS)
- Malicious bots
- Suspicious IP addresses
- Geographic locations (if configured)

Example:

A hacker sends: `https://example.com?id=' OR 1=1`

AWS WAF detects the malicious SQL injection attempt and blocks the request before it reaches the application.

Amazon CloudFront

Purpose

CloudFront is AWS's Content Delivery Network (CDN). Rather than serving every request from the main server, CloudFront caches static content at locations around the world.

Typical Cached Content

- Images
- Videos
- CSS files
- JavaScript
- PDF documents

Why use CloudFront?

Without CloudFront:

Zimbabwe Request is sent to South Africa AWS (regional centre) to fetch the image. Every request travel to the AWS Region.

With CloudFront:

Zimbabwe Request is sent to nearest edge location to fetch the image. The user receives content much faster.

Amazon S3

Purpose

Amazon S3 stores objects such as:

- Images
- Videos
- Backups
- Documents
- Static websites

CloudFront usually retrieves static files directly from S3.

Benefits

- Unlimited scalability
- Extremely durable
- Low cost
- Highly available

Virtual Private Cloud (VPC)

Purpose

A Virtual Private Cloud is an isolated network inside AWS. Everything inside the application runs within the VPC. Think of a VPC as your company's private data centre inside AWS. Inside the VPC, resources are organized into subnets.

Public Subnet

Public subnets contain resources that require Internet connectivity.

Examples include:

- NAT Gateway
- Load Balancers

These resources have public IP addresses.

Private Subnet

Private subnets contain sensitive resources.

Examples include:

- Web Servers
- Application Servers
- Databases

These servers cannot be accessed directly from the Internet. This significantly improves security.

NAT Gateway

Purpose

Private servers occasionally need Internet access.

For example:

- Operating system updates
- Downloading software packages
- Docker images
- Security patches

Instead of assigning public IP addresses, AWS provides Internet access through a NAT Gateway.

Flow:



The Internet cannot initiate connections back to the private server.

Elastic Load Balancer (ELB)

Purpose

The Elastic Load Balancer distributes incoming traffic across multiple servers.

Without a Load Balancer, 10 000 users send request to one server then the server can crash due to overload.

With a Load Balancer, 10 000 users can send request, The ELB can distribute the traffic among all the available servers. In short traffic is evenly distributed.

Benefits

- High availability
- Better performance
- Fault tolerance

Availability Zones

AWS Regions consist of multiple Availability Zones. Each Availability Zone is an independent data centre. If one Availability Zone fails, the application continues operating from another zone. This provides high availability.

Web Tier

The Web Tier contains EC2 instances responsible for handling HTTP and HTTPS requests.

Typical technologies include:

- Apache
- Nginx
- React
- Angular
- ASP.NET

The web tier receives requests from the Application Load Balancer.

Auto Scaling Group (Web Tier)

Traffic changes throughout the day. Auto Scaling automatically adjusts the number of web servers.

Example

Morning: 2 Servers

Afternoon: 8 Servers

Night: 2 Servers

Benefits:

- Handles traffic spikes
- Saves money
- Improves availability

Internal Load Balancer

The web servers forward requests to another Load Balancer. Unlike the first Load Balancer, this one is internal and not exposed to the Internet. Its purpose is to distribute requests across multiple application servers.

Application Tier

The Application Tier contains the business logic.

Examples include:

- Login authentication
- Payment processing
- Inventory calculations
- Report generation
- Customer management

The application servers process data and communicate with the database.

Amazon ElastiCache

Purpose

Frequently requested information is stored in memory.

Example:

Without Cache:

1000 similar Requests result in a 1000 Database queries.

With Cache

1000 similar Requests can use the first request's query through Redis Cache.

Benefits

- Faster response times
- Lower database load
- Improved application performance

Amazon EFS

Amazon Elastic File System provides shared storage.

Suppose Server A uploads invoice.pdf, Server B also needs access to that file. Instead of storing separate copies on each server, all servers share the same storage.

Benefits

- Centralized storage
- Automatic synchronization
- Highly scalable

Amazon RDS

Amazon RDS is a managed relational database service.

The architecture contains:

- Primary Database
- Secondary Database

This configuration is known as **Multi-AZ Deployment**. If the primary database fails, AWS automatically promotes the secondary database to become the new primary. The application continues operating with minimal interruption.

Database Subnet Group

The databases are placed inside isolated database subnets. These subnets have no Internet access. Only application servers can communicate with the databases. This improves security.

Security Layers

This architecture uses multiple layers of security.

Layer	AWS Service	Purpose
DNS	Route 53	Domain resolution
Network Protection	AWS Shield	DDoS mitigation
Application Protection	AWS WAF	Blocks malicious HTTP requests
Private Network	VPC	Isolates infrastructure
Public Access Control	Security Groups & NACLs	Controls network traffic
Data Protection	RDS Encryption, IAM	Secures sensitive data

This approach is known as **Defense in Depth**, where multiple security controls work together.

High Availability Features

This architecture is designed to remain operational even when components fail.

High availability is achieved through:

- Multiple Availability Zones
- Load Balancers
- Auto Scaling Groups
- Multi-AZ RDS
- CloudFront
- Route 53 Health Checks

Scalability Features

The application automatically adapts to changes in demand.

Scalable services include:

- Auto Scaling Groups
- Elastic Load Balancer
- Amazon CloudFront
- Amazon S3
- Amazon ElastiCache

Fault Tolerance

Fault tolerance ensures the application continues operating despite failures.

Examples:

- EC2 instance failure → Load Balancer redirects traffic to healthy instances.
- Availability Zone failure → Traffic shifts to another Availability Zone.
- Database failure → RDS promotes the standby database automatically.
- DDoS attack → AWS Shield absorbs malicious traffic.

The table below shows the equivalent for Azure if Azure is your preferred cloud service.

Layer	AWS	Azure equivalent
DNS	Route 53	Azure DNS
CDN + WAF	CloudFront + AWS WAF	Azure Front Door (built-in CDN + WAF)
DDoS	AWS Shield	Azure DDoS Protection
Load balancing	Elastic Load Balancing	Azure Load Balancer (public + internal)
Compute + autoscaling	EC2 + Auto Scaling groups	Virtual Machine Scale Sets
Network	VPC / subnets / AZs	Virtual network (VNet) / subnets / availability zones
Outbound internet	NAT gateway	NAT gateway (same concept, same name)
Relational DB	RDS (primary/secondary)	Azure SQL Database (primary + geo-replicated secondary)
Object storage	S3	Azure Blob Storage
Shared file storage	EFS	Azure Files
Caching	ElastiCache	Azure Cache for Redis